

D.T.S.S. College of Commerce (Autonomous)

Healthcare Analytics

Complete Practical Code

Practicals 1 to 6 | Chapter-wise Python Code

Class: T.Y.B.Sc. Data Science

No.	Chapter	Datasets used
1	Data Import and Exploration	diabetes_prediction_dataset.csv, heart_failure_clinical_records_dataset.csv
2	EMR Data Understanding	Medical_Appointment_No_Shows.csv
3	Data Cleaning in Healthcare	Independent_Medical_Reviews.csv
4	Categorical Data Encoding	healthcare_patient_analytics_seaborn.csv, healthcare_dataset.csv
5	Healthcare Data Analysis and Visualization	heart_failure_clinical_records_dataset.csv
6	Predictive Modelling in Healthcare	heart_failure_clinical_records_dataset.csv

Table of Contents

How to Run the Code	3
Practical 1: Data Import and Exploration	4
Q.1 Diabetes Prediction Dataset	4
Q.2 Heart Failure Clinical Records Dataset	5
Practical 2: EMR Data Understanding	7
Q.1 Medical Appointment No Shows Dataset	7
Practical 3: Data Cleaning in Healthcare	9
Q.1 Independent Medical Reviews Dataset - Data Cleaning	9
Practical 4: Categorical Data Encoding	11
Q.1 healthcare_patient_analytics_seaborn.csv	11
Q.2 Healthcare Dataset	12
Practical 5: Healthcare Data Analysis and Visualization	14
Q.1 Healthcare Data Analysis	14
Q.2 Visualizations	14
Q.3 Correlation Analysis	15
Practical 6: Predictive Modelling in Healthcare	17
Q.1 Import the dataset and prepare it	17
Q.2 Logistic Regression	17
Q.3 Decision Tree Classification	17
Q.4 Random Forest Classification	18
Q.5 Compare all three models	19

How to Run the Code

1. Install the libraries once:

```
pip install pandas numpy matplotlib seaborn scikit-learn
```

2. Keep each dataset CSV in the same folder as the Python file (or give the full path in `pd.read_csv()`).

3. Every chapter is a separate, self-contained script. Each question (Q.1, Q.2, ...) starts with a `# %%` marker, so in VS Code or Spyder you can run it cell by cell.

4. If your CSV column names differ from the ones used here (for example `Hipertension` in the `appointments` file), change the name in the code to match your file.

Practical 1: Data Import and Exploration

File: practical_1.py | Dataset(s): diabetes_prediction_dataset.csv, heart_failure_clinical_records_dataset.csv

Imports

```
import pandas as pd

# Put the CSV files in the same folder as this script (or give the full path).
```

Q.1 Diabetes Prediction Dataset

```
# a) Import the dataset
df = pd.read_csv("diabetes_prediction_dataset.csv")
print("Dataset imported successfully")

# b) Display the first five records
print("\nFirst five records:")
print(df.head())

# c) Display the last five records
print("\nLast five records:")
print(df.tail())

# d) Find the shape of the dataset
print("\nShape of the dataset (rows, columns):", df.shape)

# e) Display all column names
print("\nColumn names:")
print(list(df.columns))

# f) Display the data type of each variable
print("\nData types:")
print(df.dtypes)

# g) Generate descriptive statistics of the dataset
print("\nDescriptive statistics:")
print(df.describe())

# h) Check for missing values
print("\nMissing values in each column:")
print(df.isnull().sum())

# i) Count the number of unique values in every column
print("\nUnique values in every column:")
print(df.nunique())

# j) Frequency of diabetic and non-diabetic patients (1 = diabetic, 0 = not)
print("\nDiabetic vs non-diabetic patients:")
print(df["diabetes"].value_counts())

# k) Frequency tables for gender, hypertension, heart_disease, smoking_history
for col in ["gender", "hypertension", "heart_disease", "smoking_history"]:
    print(f"\nFrequency table for {col}:")
    print(df[col].value_counts())

# l) Find the average age of patients
print("\nAverage age of patients:", round(df["age"].mean(), 2))

# m) Find the average BMI
print("Average BMI:", round(df["bmi"].mean(), 2))

# n) Find the maximum and minimum blood glucose levels
print("Maximum blood glucose level:", df["blood_glucose_level"].max())
print("Minimum blood glucose level:", df["blood_glucose_level"].min())

# o) Find the number of male and female patients
```

```
print("\nMale and female patients:")
print(df["gender"].value_counts())

# p) Number of patients having hypertension
print("\nPatients having hypertension:", (df["hypertension"] == 1).sum())

# q) Number of patients having heart disease
print("Patients having heart disease:", (df["heart_disease"] == 1).sum())

# r) Find the average HbA1c level
print("Average HbA1c level:", round(df["HbA1c_level"].mean(), 2))
```

Q.2 Heart Failure Clinical Records Dataset

```
# a) Import the dataset
hf = pd.read_csv("heart_failure_clinical_records_dataset.csv")
print("Dataset imported successfully")

# b) Display the first five records
print("\nFirst five records:")
print(hf.head())

# c) Display the last five records
print("\nLast five records:")
print(hf.tail())

# d) Find the shape of the dataset
print("\nShape of the dataset (rows, columns):", hf.shape)

# e) Display all column names
print("\nColumn names:")
print(list(hf.columns))

# f) Display the data type of each variable
print("\nData types:")
print(hf.dtypes)

# g) Generate descriptive statistics of the dataset
print("\nDescriptive statistics:")
print(hf.describe())

# h) Check for missing values
print("\nMissing values in each column:")
print(hf.isnull().sum())

# i) Count the number of unique values in every column
print("\nUnique values in every column:")
print(hf.nunique())

# j) Frequency of the target variable DEATH_EVENT (1 = died, 0 = survived)
print("\nFrequency of DEATH_EVENT:")
print(hf["DEATH_EVENT"].value_counts())

# k) Find the average age of the patients
print("\nAverage age of patients:", round(hf["age"].mean(), 2))

# l) Find the average ejection fraction
print("Average ejection fraction:", round(hf["ejection_fraction"].mean(), 2))

# m) Find the maximum and minimum serum creatinine levels
print("Maximum serum creatinine:", hf["serum_creatinine"].max())
print("Minimum serum creatinine:", hf["serum_creatinine"].min())
# n) Number of male and female patients (sex: 1 = male, 0 = female)
print("\nMale and female patients:")
print(hf["sex"].map({1: "Male", 0: "Female"}).value_counts())

# o) Number of patients having diabetes
print("\nPatients having diabetes:")
```

```
print(hf["diabetes"].map({1: "Diabetic", 0: "Non-diabetic"}).value_counts())

# p) Number of patients having high blood pressure
print("\nPatients having high blood pressure:")
print(hf["high_blood_pressure"].map({1: "High BP", 0: "Normal BP"}).value_counts())

# q) Find the average platelet count
print("\nAverage platelet count:", round(hf["platelets"].mean(), 2))

# r) Find the average serum sodium level
print("Average serum sodium level:", round(hf["serum_sodium"].mean(), 2))

# s) Number of smokers and non-smokers
print("\nSmokers and non-smokers:")
print(hf["smoking"].map({1: "Smoker", 0: "Non-smoker"}).value_counts())

# t) Number of patients with and without anaemia
print("\nPatients with and without anaemia:")
print(hf["anaemia"].map({1: "With anaemia", 0: "Without anaemia"}).value_counts())
```

Practical 2: EMR Data Understanding

File: practical_2.py | Dataset(s): Medical_Appointment_No_Shows.csv

Imports

```
import pandas as pd
```

Q.1 Medical Appointment No Shows Dataset

```
# a) Import the dataset
df = pd.read_csv("Medical_Appointment_No_Shows.csv")
print("Dataset imported successfully")

# Tidy a few known column-name quirks of this dataset (only if present)
df = df.rename(columns={
    "Hipertension": "Hypertension",
    "Handcap": "Handicap",
    "No-show": "No_show",
})

# b) Display the first five records
print("\nFirst five records:")
print(df.head())

# c) Display the last five records
print("\nLast five records:")
print(df.tail())

# d) Display the shape of the dataset
print("\nShape of the dataset (rows, columns):", df.shape)

# e) Display all column names
print("\nColumn names:")
print(list(df.columns))

# f) Display the data type of each variable
print("\nData types:")
print(df.dtypes)

# g) Identify the demographic variables
demographic = ["PatientId", "Gender", "Age", "Neighbourhood", "Scholarship"]
print("\nDemographic variables:", demographic)

# h) Identify the clinical variables
clinical = ["Hypertension", "Diabetes", "Alcoholism", "Handicap"]
print("\nClinical variables:", clinical)

# i) Identify the administrative variables
administrative = ["AppointmentID", "ScheduledDay", "AppointmentDay", "SMS_received"]
print("\nAdministrative variables:", administrative)

# j) Identify the outcome variable
outcome = ["No_show"]
print("\nOutcome variable:", outcome)

# k) Frequency of male and female patients
print("\nMale and female patients:")
print(df["Gender"].value_counts())

# l) Frequency of patients having hypertension
print("\nPatients having hypertension (1 = yes, 0 = no):")
print(df["Hypertension"].value_counts())

# m) Frequency of patients having diabetes
print("\nPatients having diabetes (1 = yes, 0 = no):")
```

```
print(df["Diabetes"].value_counts())

# n) Frequency of patients with alcoholism
print("\nPatients with alcoholism (1 = yes, 0 = no):")
print(df["Alcoholism"].value_counts())

# o) Frequency of patients receiving SMS reminders
print("\nPatients receiving SMS reminders (1 = yes, 0 = no):")
print(df["SMS_received"].value_counts())

# p) Frequency of patients who attended and missed their appointments (No-show)
print("\nNo-show frequency (No = attended, Yes = missed):")
print(df["No_show"].value_counts())

# q) Display the min, max and average age of patients
print("\nAge of patients:")
print("Minimum age :", df["Age"].min())
print("Maximum age :", df["Age"].max())
print("Average age :", round(df["Age"].mean(), 2))

# r) Find the number of unique neighbourhoods
print("\nNumber of unique neighbourhoods:", df["Neighbourhood"].nunique())

# s) Count the number of unique patients
print("Number of unique patients:", df["PatientId"].nunique())

# t) Create a table showing each variable and classify it
classification = pd.DataFrame({
    "Variable": demographic + clinical + administrative + outcome,
    "Category": (["Demographic"] * len(demographic)
                 + ["Clinical"] * len(clinical)
                 + ["Administrative"] * len(administrative)
                 + ["Outcome"] * len(outcome)),
})
print("\nVariable classification table:")
print(classification.to_string(index=False))
```

Practical 3: Data Cleaning in Healthcare

File: practical_3.py | Dataset(s): Independent_Medical_Reviews.csv

Imports

```
import pandas as pd
```

Q.1 Independent Medical Reviews Dataset - Data Cleaning

```
# a) Import the dataset
df = pd.read_csv("Independent_Medical_Reviews.csv")
print("Dataset imported successfully")

# b) Display the first five records
print("\nFirst five records:")
print(df.head())

# c) Display the number of rows and columns before cleaning
print("\nRows and columns before cleaning:", df.shape)

# d) Check for missing values in each column
print("\nMissing values in each column:")
print(df.isnull().sum())

# e) Find the total number of missing values in the dataset
print("\nTotal missing values in the dataset:", df.isnull().sum().sum())

# f) Check for duplicate records
print("\nDuplicate records present?:", df.duplicated().any())

# g) Find the total number of duplicate records
print("Total number of duplicate records:", df.duplicated().sum())

# h) Remove duplicate records, if any
df = df.drop_duplicates()
print("Shape after removing duplicates:", df.shape)

# i) Check the unique values in the Diagnosis Category column
print("\nUnique values in Diagnosis Category:")
print(df["Diagnosis Category"].unique())

# j) Check the unique values in the Treatment Category column
print("\nUnique values in Treatment Category:")
print(df["Treatment Category"].unique())

# k) Check the unique values in the Determination column
print("\nUnique values in Determination:")
print(df["Determination"].unique())

# l) Check the unique values in the Patient Gender column
print("\nUnique values in Patient Gender:")
print(df["Patient Gender"].unique())

# m) Identify the columns containing missing values
missing_cols = df.columns[df.isnull().any()].tolist()
print("\nColumns containing missing values:", missing_cols)
# n) Replace missing values in categorical columns with "Unknown"
categorical_cols = df.select_dtypes(include="object").columns
df[categorical_cols] = df[categorical_cols].fillna("Unknown")

# o) Verify that the missing values have been handled
print("\nMissing values after handling:")
print(df.isnull().sum())

# p) Convert the Report Year column to an appropriate numeric data type, if required
```

```
print("\nData type of Report Year before conversion:", df["Report Year"].dtype)
df["Report Year"] = pd.to_numeric(df["Report Year"], errors="coerce")
# Any value that could not be converted becomes NaN; drop such rows if present
df = df.dropna(subset=["Report Year"])
df["Report Year"] = df["Report Year"].astype(int)
print("Data type of Report Year after conversion :", df["Report Year"].dtype)

# q) Display the number of rows and columns after cleaning
print("\nRows and columns after cleaning:", df.shape)

# r) Verify that the cleaned dataset contains no duplicate records
print("Duplicate records in cleaned dataset:", df.duplicated().sum())

# s) Display the first five records of the cleaned dataset
print("\nFirst five records of the cleaned dataset:")
print(df.head())

# t) Save the cleaned dataset
df.to_csv("cleaned_independent_medical_reviews.csv", index=False)
print("\nCleaned dataset saved as cleaned_independent_medical_reviews.csv")
```

Practical 4: Categorical Data Encoding

File: practical_4.py | Dataset(s): healthcare_patient_analytics_seaborn.csv, healthcare_dataset.csv

Imports

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

Q.1 healthcare_patient_analytics_seaborn.csv

```
# a) Import the dataset
df1 = pd.read_csv("healthcare_patient_analytics_seaborn.csv")
print("Dataset imported successfully")

# b) Display the first five records
print("\nFirst five records:")
print(df1.head())

# c) Display the column names and data types
print("\nColumn names:")
print(list(df1.columns))
print("\nData types:")
print(df1.dtypes)

# d) Identify the categorical variables in the dataset
cat_cols = df1.select_dtypes(include=["object", "category"]).columns.tolist()
print("\nCategorical variables:", cat_cols)

# e) Display the unique values of the gender variable
print("\nUnique values of gender:", df1["gender"].unique())

# f) Display the unique values of the age_group variable
print("Unique values of age_group:", df1["age_group"].unique())

# g) Display the unique values of the visit_type variable
print("Unique values of visit_type:", df1["visit_type"].unique())

# h) Apply Label Encoding to the gender variable
le = LabelEncoder()
df1["gender_encoded"] = le.fit_transform(df1["gender"])

# i) Display the encoded values of gender
print("\nOriginal vs encoded gender:")
print(df1[["gender", "gender_encoded"]].drop_duplicates().to_string(index=False))

# j) Apply Label Encoding to the visit_type variable
le_visit = LabelEncoder()
df1["visit_type_encoded"] = le_visit.fit_transform(df1["visit_type"])

# k) Display the encoded values of visit_type
print("\nOriginal vs encoded visit_type:")
print(df1[["visit_type", "visit_type_encoded"]].drop_duplicates().to_string(index=False))

# l) Apply One-Hot Encoding to the age_group variable
age_dummies = pd.get_dummies(df1["age_group"], prefix="age_group", dtype=int)

# m) Display the newly created dummy variables for age_group
print("\nDummy variables for age_group:")
print(age_dummies.head())
df1 = pd.concat([df1, age_dummies], axis=1)

# n) Apply One-Hot Encoding to the treatment_type variable
treat_dummies = pd.get_dummies(df1["treatment_type"], prefix="treatment_type", dtype=int)

# o) Display the newly created dummy variables for treatment_type
```

```
print("\nDummy variables for treatment_type:")
print(treat_dummies.head())
df1 = pd.concat([df1, treat_dummies], axis=1)

# Drop the original text columns that have now been encoded
df1_encoded = df1.drop(columns=["gender", "visit_type", "age_group", "treatment_type"])

# p) Display the first five records of the encoded dataset
print("\nFirst five records of the encoded dataset:")
print(df1_encoded.head())

# q) Display the data types of all variables after encoding
print("\nData types after encoding:")
print(df1_encoded.dtypes)

# r) Save the final encoded dataset
df1_encoded.to_csv("encoded_healthcare_patient_analytics.csv", index=False)
print("\nSaved as encoded_healthcare_patient_analytics.csv")
```

Q.2 Healthcare Dataset

```
# a) Import the dataset using Pandas
df2 = pd.read_csv("healthcare_dataset.csv")
print("Dataset imported successfully")

# b) Display the first five records
print("\nFirst five records:")
print(df2.head())

# c) Display the column names and data types
print("\nColumn names:")
print(list(df2.columns))
print("\nData types:")
print(df2.dtypes)

# d) Identify and display all categorical variables
cat_cols2 = df2.select_dtypes(include=["object", "category"]).columns.tolist()
print("\nCategorical variables:", cat_cols2)

# e) Display the unique values of the Gender variable
print("\nUnique values of Gender:", df2["Gender"].unique())

# f) Display the unique values of the Blood Type variable
print("Unique values of Blood Type:", df2["Blood Type"].unique())

# g) Display the unique values of the Medical Condition variable
print("Unique values of Medical Condition:", df2["Medical Condition"].unique())

# h) Display the unique values of the Medication variable
print("Unique values of Medication:", df2["Medication"].unique())

# i) Display the unique values of the Admission Type variable
print("Unique values of Admission Type:", df2["Admission Type"].unique())

# j) Display the unique values of the Test Results variable
print("Unique values of Test Results:", df2["Test Results"].unique())

# Remember the number of columns before encoding (used in step x)
cols_before = df2.shape[1]

# k) Apply Label Encoding to the Gender variable
le_gender = LabelEncoder()
df2["Gender_Encoded"] = le_gender.fit_transform(df2["Gender"])

# l) Display the original and encoded values of the Gender variable
print("\nOriginal vs encoded Gender:")
print(df2[["Gender", "Gender_Encoded"]].drop_duplicates().to_string(index=False))

# m) Apply Label Encoding to the Medication variable
```

```
le_med = LabelEncoder()
df2["Medication_Encoded"] = le_med.fit_transform(df2["Medication"])

# n) Display the original and encoded values of the Medication variable
print("\nOriginal vs encoded Medication:")
print(df2[["Medication", "Medication_Encoded"]].drop_duplicates().to_string(index=False))

# o) Apply One-Hot Encoding to the Blood Type variable
blood_dummies = pd.get_dummies(df2["Blood Type"], prefix="Blood Type", dtype=int)

# p) Display the newly created dummy variables for Blood Type
print("\nDummy variables for Blood Type:")
print(blood_dummies.head())

# q) Apply One-Hot Encoding to the Medical Condition variable
cond_dummies = pd.get_dummies(df2["Medical Condition"], prefix="Medical Condition", dtype=int)

# r) Display the newly created dummy variables for Medical Condition
print("\nDummy variables for Medical Condition:")
print(cond_dummies.head())

# s) Apply One-Hot Encoding to the Admission Type variable
adm_dummies = pd.get_dummies(df2["Admission Type"], prefix="Admission Type", dtype=int)

# t) Display the newly created dummy variables for Admission Type
print("\nDummy variables for Admission Type:")
print(adm_dummies.head())

# u) Apply One-Hot Encoding to the Test Results variable
test_dummies = pd.get_dummies(df2["Test Results"], prefix="Test Results", dtype=int)

# Build the final encoded dataset (replace original text columns with encoded ones)
encoded = pd.concat([df2, blood_dummies, cond_dummies, adm_dummies, test_dummies], axis=1)
encoded = encoded.drop(columns=["Gender", "Medication", "Blood Type",
                                "Medical Condition", "Admission Type", "Test Results"])

# v) Display the first five records of the final encoded dataset
print("\nFirst five records of the final encoded dataset:")
print(encoded.head())

# w) Display the data types of all variables after encoding
print("\nData types after encoding:")
print(encoded.dtypes)

# x) Compare the number of columns before and after encoding
print("\nNumber of columns before encoding:", cols_before)
print("Number of columns after encoding :", encoded.shape[1])
# y) Save the encoded dataset
encoded.to_csv("encoded_healthcare_dataset.csv", index=False)
print("\nSaved as encoded_healthcare_dataset.csv")
```

Practical 5: Healthcare Data Analysis and Visualization

File: practical_5.py | Dataset(s): heart_failure_clinical_records_dataset.csv

Imports

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Q.1 Healthcare Data Analysis

```
# a) Import the dataset (shown as part of the first observations below)
df = pd.read_csv("heart_failure_clinical_records_dataset.csv")
print("Dataset imported successfully")

# a) Display the first five observations
print("\nFirst five observations:")
print(df.head())

# b) Display the number of rows and columns
print("\nNumber of rows   :", df.shape[0])
print("Number of columns:", df.shape[1])

# c) Display the column names and data types
print("\nColumn names:")
print(list(df.columns))
print("\nData types:")
print(df.dtypes)

# d) Display summary statistics of all numerical variables
print("\nSummary statistics:")
print(df.describe())

# e) Check for missing values in the dataset
print("\nMissing values in each column:")
print(df.isnull().sum())

# f) Calculate the average Age, Ejection Fraction and Serum Creatinine
print("\nAverage Age           :", round(df["age"].mean(), 2))
print("Average Ejection Fraction :", round(df["ejection_fraction"].mean(), 2))
print("Average Serum Creatinine  :", round(df["serum_creatinine"].mean(), 2))

# g) Display the frequency of patients based on DEATH_EVENT
print("\nFrequency of DEATH_EVENT (0 = survived, 1 = died):")
print(df["DEATH_EVENT"].value_counts())
```

Q.2 Visualizations

```
sns.set_style("whitegrid")

# a) Histogram of Age to study the age distribution of patients
plt.figure(figsize=(8, 5))
plt.hist(df["age"], bins=15, color="steelblue", edgecolor="black")
plt.title("Age Distribution of Patients")
plt.xlabel("Age")
plt.ylabel("Number of Patients")
plt.show()

# b) Bar chart of Male and Female patients (sex: 1 = male, 0 = female)
plt.figure(figsize=(6, 5))
sex_counts = df["sex"].map({1: "Male", 0: "Female"}).value_counts()
```

```

plt.bar(sex_counts.index, sex_counts.values, color=["#4C72B0", "#DD8452"])
plt.title("Male and Female Patients")
plt.xlabel("Sex")
plt.ylabel("Number of Patients")
plt.show()

# c) Bar chart showing patients with and without Diabetes
plt.figure(figsize=(6, 5))
diab_counts = df["diabetes"].map({1: "With Diabetes", 0: "Without Diabetes"}).value_counts()
plt.bar(diab_counts.index, diab_counts.values, color=["#55A868", "#C44E52"])
plt.title("Patients With and Without Diabetes")
plt.ylabel("Number of Patients")
plt.show()

# d) Bar chart showing patients with and without High Blood Pressure
plt.figure(figsize=(6, 5))
bp_counts = df["high_blood_pressure"].map(
    {1: "With High BP", 0: "Without High BP"}).value_counts()
plt.bar(bp_counts.index, bp_counts.values, color=["#8172B3", "#CCB974"])
plt.title("Patients With and Without High Blood Pressure")
plt.ylabel("Number of Patients")
plt.show()

# e) Pie chart showing the proportion of patients who survived and died
plt.figure(figsize=(6, 6))
death_counts = df["DEATH_EVENT"].map({0: "Survived", 1: "Died"}).value_counts()
plt.pie(death_counts.values, labels=death_counts.index, autopct="%1.1f%%",
        colors=["#55A868", "#C44E52"], startangle=90)
plt.title("Survived vs Died")
plt.show()

# f) Boxplot of Serum Creatinine to identify spread and possible outliers
plt.figure(figsize=(6, 5))
sns.boxplot(y=df["serum_creatinine"], color="lightcoral")
plt.title("Boxplot of Serum Creatinine")
plt.show()

# g) Boxplot of Ejection Fraction
plt.figure(figsize=(6, 5))
sns.boxplot(y=df["ejection_fraction"], color="lightgreen")
plt.title("Boxplot of Ejection Fraction")
plt.show()

# h) Histogram of Serum Sodium
plt.figure(figsize=(8, 5))
plt.hist(df["serum_sodium"], bins=15, color="mediumpurple", edgecolor="black")
plt.title("Distribution of Serum Sodium")
plt.xlabel("Serum Sodium")
plt.ylabel("Number of Patients")
plt.show()

```

Q.3 Correlation Analysis

```
# a) Select the numerical variables from the dataset
num_df = df.select_dtypes(include=[np.number])
print("\nNumerical variables:", list(num_df.columns))

# b) Calculate the correlation matrix
corr = num_df.corr()
print("\nCorrelation matrix:")
print(corr.round(2))

# c) Display the correlation matrix using a heatmap
plt.figure(figsize=(12, 9))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()

# d) Identify variables with relatively higher positive or negative
#     correlation with DEATH_EVENT
death_corr = corr["DEATH_EVENT"].drop("DEATH_EVENT").sort_values(ascending=False)
print("\nCorrelation of each variable with DEATH_EVENT:")
print(death_corr.round(3))
print("\nStrongest positive correlation :", death_corr.idxmax(),
      round(death_corr.max(), 3))
print("Strongest negative correlation :", death_corr.idxmin(),
      round(death_corr.min(), 3))

# e) Scatter plot of Age vs Serum Creatinine
plt.figure(figsize=(8, 5))
plt.scatter(df["age"], df["serum_creatinine"], alpha=0.7, color="teal")
plt.title("Age vs Serum Creatinine")
plt.xlabel("Age")
plt.ylabel("Serum Creatinine")
plt.show()

# f) Scatter plot of Ejection Fraction vs Serum Creatinine
plt.figure(figsize=(8, 5))
plt.scatter(df["ejection_fraction"], df["serum_creatinine"], alpha=0.7, color="darkorange")
plt.title("Ejection Fraction vs Serum Creatinine")
plt.xlabel("Ejection Fraction")
plt.ylabel("Serum Creatinine")
plt.show()
```

Practical 6: Predictive Modelling in Healthcare

File: practical_6.py | Dataset(s): heart_failure_clinical_records_dataset.csv

Imports

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay)
```

Q.1 Import the dataset and prepare it

```
df = pd.read_csv("heart_failure_clinical_records_dataset.csv")
print("Dataset imported successfully")
print(df.head())

# a) Separate the independent variables and the target variable DEATH_EVENT
X = df.drop("DEATH_EVENT", axis=1)
y = df["DEATH_EVENT"]
print("\nIndependent variables shape:", X.shape)
print("Target variable shape      :", y.shape)

# b) Divide the dataset into training and testing sets in the ratio 80:20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
print("\nTraining set size:", X_train.shape[0])
print("Testing set size :", X_test.shape[0])
```

Q.2 Logistic Regression

```
# a) Create a Logistic Regression model using the training data
log_model = LogisticRegression(max_iter=5000)
log_model.fit(X_train, y_train)

# b) Predict DEATH_EVENT for the test data
y_pred_log = log_model.predict(X_test)
print("\nPredictions (first 10):", y_pred_log[:10])

# c) Display the confusion matrix
cm_log = confusion_matrix(y_test, y_pred_log)
print("\nConfusion matrix (Logistic Regression):")
print(cm_log)
ConfusionMatrixDisplay(cm_log).plot(cmap="Blues")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()

# d) Calculate the Accuracy, Precision, Recall and F1-Score
acc_log = accuracy_score(y_test, y_pred_log)
prec_log = precision_score(y_test, y_pred_log)
rec_log = recall_score(y_test, y_pred_log)
f1_log = f1_score(y_test, y_pred_log)
print(f"\nAccuracy   : {acc_log:.4f}")
print(f"Precision   : {prec_log:.4f}")
print(f"Recall      : {rec_log:.4f}")
print(f"F1-Score    : {f1_log:.4f}")
```

Q.3 Decision Tree Classification

```
# a) Create a Decision Tree classifier using the training data
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# b) Predict DEATH_EVENT for the test data
y_pred_dt = dt_model.predict(X_test)
print("\nPredictions (first 10):", y_pred_dt[:10])

# c) Display the confusion matrix
cm_dt = confusion_matrix(y_test, y_pred_dt)
print("\nConfusion matrix (Decision Tree):")
print(cm_dt)
ConfusionMatrixDisplay(cm_dt).plot(cmap="Greens")
plt.title("Confusion Matrix - Decision Tree")
plt.show()

# d) Calculate the Accuracy, Precision, Recall and F1-Score
acc_dt = accuracy_score(y_test, y_pred_dt)
prec_dt = precision_score(y_test, y_pred_dt)
rec_dt = recall_score(y_test, y_pred_dt)
f1_dt = f1_score(y_test, y_pred_dt)
print(f"\nAccuracy : {acc_dt:.4f}")
print(f"Precision : {prec_dt:.4f}")
print(f"Recall : {rec_dt:.4f}")
print(f"F1-Score : {f1_dt:.4f}")
```

Q.4 Random Forest Classification

```
# a) Create a Random Forest classifier using the training data
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# b) Predict DEATH_EVENT for the test data
y_pred_rf = rf_model.predict(X_test)
print("\nPredictions (first 10):", y_pred_rf[:10])

# c) Display the confusion matrix
cm_rf = confusion_matrix(y_test, y_pred_rf)
print("\nConfusion matrix (Random Forest):")
print(cm_rf)
ConfusionMatrixDisplay(cm_rf).plot(cmap="Oranges")
plt.title("Confusion Matrix - Random Forest")
plt.show()

# d) Calculate the Accuracy, Precision, Recall and F1-Score
acc_rf = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf)
rec_rf = recall_score(y_test, y_pred_rf)
f1_rf = f1_score(y_test, y_pred_rf)
print(f"\nAccuracy : {acc_rf:.4f}")
print(f"Precision : {prec_rf:.4f}")
print(f"Recall : {rec_rf:.4f}")
print(f"F1-Score : {f1_rf:.4f}")

# e) Display the feature importance of the variables
importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": rf_model.feature_importances_,
}).sort_values("Importance", ascending=False)
print("\nFeature importance:")
print(importance.to_string(index=False))

plt.figure(figsize=(9, 6))
plt.barh(importance["Feature"], importance["Importance"], color="seagreen")
plt.gca().invert_yaxis()
plt.title("Random Forest - Feature Importance")
plt.xlabel("Importance")
plt.show()
```

Q.5 Compare all three models

```
comparison = pd.DataFrame({
    "Model": ["Logistic Regression", "Decision Tree", "Random Forest"],
    "Accuracy": [acc_log, acc_dt, acc_rf],
    "Precision": [prec_log, prec_dt, prec_rf],
    "Recall": [rec_log, rec_dt, rec_rf],
    "F1-Score": [f1_log, f1_dt, f1_rf],
}).round(4)
print("\nModel comparison:")
print(comparison.to_string(index=False))

best = comparison.loc[comparison["F1-Score"].idxmax(), "Model"]
print("\nBest model based on F1-Score:", best)
```